数学与 CS 问答笔记

answer-note

按日归档的问答笔记，承接 2026-10-15 的 TDD 循环一节：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 16, 2026

Contents

1. 当日起点：项目状态与工具状态 3
2. 考研并行下做项目：动手之外的“证据工程” 4
2.1. 费曼直觉 4
2.2. 硬核机制 4
2.2.1. 八个动作：一张总表 4
2.2.2. 四个核心动作（每次实践顺手做） 5
2.2.3. 三个进阶动作（里程碑级，低频） 7
2.2.4. 考研特供：与 408 双向绑定 8
2.2.5. 时间盒：怎么塞进考研日程 9
2.3. 陷阱与易错 9
2.4. 关键点 9
2.5. 变式检验 9
2.6. 同类归类 10
2.7. 本次操作与当前状态 10
Index of Listings 12

1. 当日起点：项目状态与工具状态

承接 10-15 那一节收尾时的状态：TDD 的循环已经在沙盒里完整预演（红 → 绿），欠仓库的就是两步改动。今天先复核了一遍现场：

项目项	当前状态	今天实测
`internal/db/kv.go`	`Del` 仍只有签名（第 29 行），整个包编译不过	`go build ./...` 报 `missing function body`
`internal/db/kv_test.go`	第 22、31 行的两处“未命中”契约在等实现转绿	白盒测试先行，保持“红”
`scratch/kvprobe/`	依赖 `internal/db`，被 `Del` 拖住	没能跑
10-15 的两条待办	① 补 `Del` 函数体；② `Get` 未命中改 `nil, false, nil`（并删 `import "errors"`）	均未做

📌 **核心要点** 上一条未收口的问题（10-15《在 `kv.go` 里落地 TDD》）：**循环停在“红”**——沙盒里已预演全程，落回仓库只差上面两步；做完后 `go test -count=1 ./internal/db/` 应输出 `ok`。

⚠️ **陷阱预警** 今天的问题偏方法论（见下一节），**不依赖你先修绿代码**；但注意：那两步改动修绿的过程，恰好是本文“③ 故障档案 + ① 决策日志”的第一份素材——修的时候顺手记，5 分钟成本。

2. 考研并行下做项目：动手之外的“证据工程”

2.1. 费曼直觉

面试官手里的简历是“声明”，面试是“举证”环节。

你当前的情况，要把两件事拆开看：

1. **动手实践**（你已经在做）产出的是“经历”——我写过 `kv.go`、我跑通过测试；
2. **面试脱颖而出**需要的是“证据”——能当场证明这段经历是**你**做的、**你**想清楚了的可验证痕迹。

为什么这个差额值钱：面试官（以及考研复试的导师组）见过无数份简历上写着“用 Go 实现了一个 KV 存储”。这句话本身的信息量接近于零——它只说明你**有**这件事。他真正在搜索的是三个信号：

信号	他真正想确认的	他会怎么问
真实性	这代码真是你写的、你真跑过	这一行为什么这样写？你跑过什么实验？
深度	知其所以然：取舍、边界、失败面	为什么不用 Y？坏了会怎样？
可复述性	3 分钟能讲清、并接得住追问	给我讲讲你的项目（然后连问 20 分钟）

于是可以这样记：**项目是矿山，动手是采矿，“证据动作”是炼金属**。只挖不炼，面试时手里只有一句“我做了一个 KV 存储”——这是所有人都会说的话。

一句话版本：**脱颖而出 = 把“我做过 X”升级成“我能用决策、数字、失败案例证明 X 是我做的、并且我懂它”**。

下面八个动作，全部是“炼金属”的动作——它们不改你的项目路线，只改你在路线上的**记录方式**。

2.2. 硬核机制

2.2.1. 八个动作：一张总表

按性价比排序，单次成本 5–20 分钟，不需要新开战线：

动作	具体做法（成本）	信号 → 本项目落地
① 决策日志	每个设计选择记 5 行：决策/备选/约束/代价/何时重选（5 分钟）	深度：`Get` 未命中为何用 `ok` 不用 `error`（10-13 已裁决）
② 量化数字	基准测试 + 覆盖率 + 竞态检测，记 ns/op 与分配数（一次性搭好）	真实：命中路径约百 ns/op、0 allocs/op 进简历
③ 故障档案	每个 bug 记 6 行：症状/复现/定位过程/根因/修复/防复发（10 分钟）	韧性：“最难的问题”有细节可讲（`vet` 静默事件）
④ 破坏实验	空/nil/超长/并发输入矩阵，维护失效模式表（每模块 20 分钟）	严谨：“坏起来什么样”说得出来

⑤ 生产对照	读一个真实系统（Badger/bbolt/etcd），做对照表（1–2 小时，可推迟）	视野：区分“课程作业”与“工程能力”
⑥ 叙事卡	每里程碑一张 STAR 卡 + 30 秒/3 分钟两版讲稿（15 分钟）	表达：项目变成讲得清的故事
⑦ 公开痕迹	提交粒度与 message、README、把笔记 PDF 挂进仓库（渐进）	可信：面试官会点开 GitHub 看
⑧ 考研绑定	408 考点 ↔ 项目模块 ↔ 面试考点，三列映射表（见后）	双赢：复习即项目、项目即复习

2.2.2. 四个核心动作（每次实践顺手做）

① **决策日志（decision log）**——把“为什么选 A 不选 B”写下来，只要 5 行：

1. 决策：一句话；
2. 备选：当时考虑过的 2–3 个方案；
3. 约束：在什么条件下做的决定；
4. 代价：选它付了什么（复杂度 / 性能 / 工期）；
5. 何时重选：什么信号出现会推翻它。

为什么值钱：面试官深挖项目时问的不是“你写了什么”，而是“为什么这么写”——现场回忆等于现编，一股“事后诸葛亮”味；掏出日志则有完整的过程感。

本项目落地：你其实已经写过 ADR 了，只是没起这个名字——10-13 对“`Get` 未命中返回 `nil, false, nil` 而不是 `error`”的裁决、09-30 对 test/db 目录组织的取舍、10-15 对 TDD 循环的复盘，全是现成条目。这个做法有标准名字：架构决策记录（Architecture Decision Record，ADR）[1]。从今天起，每个裁决当场留 5 行，先统一放进 `note/`，等模块齐了再抽取到仓库 `docs/decisions.md`。

② **量化数字（benchmark / coverage / race）**——让“我做得好”有数。三件工具各管一段：

1. **基准测试**（benchmark）：`go test -bench` 加 `-benchmem`，拿到两个关键数——每操作耗时（ns/op）与每操作分配（allocs/op）；
2. **覆盖率**（coverage）：`go test -cover`，看测试盲区在哪里；
3. **竞态检测**（race detector）：`go test -race`，将来加并发能力时必测。

等 `Del` 修绿后，`Get` / `Set` 就是你的第一批基准。模板（放 `internal/db/`，与实现同包、白盒）：

[1] 来源：Michael Nygard, “Documenting Architecture Decisions”（2011）；本笔记取其“决策-背景-后果”三段式，缩减为 5 行。

```go
// internal/db/bench_test.go——基准测试模板
package db

import (
  "fmt"
  "testing"
)

func BenchmarkKVGetHit(b *testing.B) {
  kv := KV{}
  if err := kv.Open(); err ≠ nil {
    b.Fatal(err)
  }
  defer kv.Close()

  // 预生成 keys：准备开销不进入计时窗口
  keys := make([][]byte, 1000)
  for i := range keys {
    keys[i] = []byte(fmt.Sprintf("key-%04d", i))
    if _, err := kv.Set(keys[i], []byte("value")); err ≠ nil {
      b.Fatal(err)
    }
  }

  b.ResetTimer()
  for i := 0; i < b.N; i++ {
    if _, _, err := kv.Get(keys[i%len(keys)]); err ≠ nil {
      b.Fatal(err)
    }
  }
}
```

Listing 1: 基准测试模板：`b.ResetTimer` 之前只做数据准备；`keys` 预生成而不是在循环里 `fmt.Sprintf`，否则测到的是“Sprintf + 查找”而不是“查找”

```
$ go test -bench . -benchmem -count=1 ./internal/db/
goos: linux
goarch: amd64
pkg: github.com/zz-cpp/db-go/internal/db
cpu: （此处是你的 CPU 型号）
BenchmarkKVGetHit-16        3200000        380.5 ns/op       0 B/op       0 allocs/op
PASS
ok      github.com/zz-cpp/db-go/internal/db      1.4s

$ go test -race -count=1 ./internal/db/
ok      github.com/zz-cpp/db-go/internal/db      2.1s
```

Listing 2: 运行形状示意（数字以你机器实测为准）：学会读三列——`ns/op`（每次操作耗时）、`B/op`（每次操作字节数）、`allocs/op`（每次操作分配次数）；最后一行是竞态检测通过的样子

两个陷阱先说清：第一，准备开销要用 `b.ResetTimer` 隔开——见上面注释；第二，数字要能**解释**：“为什么这个版本比上个版本快 3 倍”比“我有个 380 的数字”值钱得多——测量是为了发现与解释，不是为了刷数字。

③ **故障档案（incident log）**——每个 bug 记 6 行：症状 / 复现步骤 / 定位过程（试了什么、排除了什么）/ 根因 / 修复 / 防复发。

为什么值钱：“讲讲你解决过最难（印象最深）的问题”是面试必问题。人对细节的记忆只有几天保质期——没有档案，你只能讲空泛版本。对照一下同一件事的两种讲法：

1. **空泛版**：“遇到过编译错误，调了很久，最后解决了。”（信息量为零）
2. **档案版**：“`go vet` 对一个缺函数体的声明静默通过、退出码 0，看起来一切正常。我先确认不是测试代码自身的问题，再写了一个最小对照包（只有一个缺函数体的文件），确认 `vet` 根本不负责这类检查——最后把'红'的信号源切换成 `go test` / `go build`。”

后半段就是 10-15 那节的实测故事——它已经躺在你的笔记里，只差贴上“故障档案 1”的标签。

本项目已有素材：`vet` 静默事件（10-15）、`testify` 的 `BrokenImport`（09-30）、`Del` 的语义三值设计（10-13）。

④ **破坏实验（failure injection）**——主动把东西弄坏，维护一张“失效模式表”（输入形态 × 期望行为 × 实际行为 × 防御位置）：

场景	试什么	你要能回答
空与 nil	`Set(nil, nil)`、空 `[]byte{}`	崩溃？静默吞掉？契约怎么定？
超长	极大的 key / value	内存如何增长？要不要上限？
并发	多 goroutine 同时 `Set` / `Get`，跑 `go test -race`	现在安全吗？将来靠什么机制变安全？
中断	写到一半杀进程（等有文件持久化之后）	数据会坏成什么样？如何恢复？

为什么值钱：别人演示顺利路径（happy path），你能讲“它什么时候会坏、坏成什么样、我用什么防住”——这是工程成熟度的直接证据；同时也是 ③ 的素材来源。

2.2.3. 三个进阶动作（里程碑级，低频）

⑤ **生产实现对照**——挑一个真实系统读一读，做一张“我的实现 vs 生产实现”对照表：

1. 候选：Badger（纯 Go 的 LSM 树 KV）、bbolt（etcd 用的 B+ 树存储）、etcd（看它如何用 Raft，量力而行）、SQLite（设计文档质量极高）；
2. 维度模板：数据组织 / 索引 / 持久化 / 并发 / 崩溃恢复 ；
3. 考研期只要求 README 与设计文档级别，深读留到初试后。

为什么值钱：面试官能分辨“只做过课程作业”和“知道真实系统如何解决同一问题”——后者叫工程视野，是简历上最难造假的东西。

⑥ **叙事卡（STAR card）**——每个里程碑写一张卡：背景（Situation）/ 任务（Task）/ 行动（Action，写“我做的关键决策”）/ 结果（Result，放数字）/ 复盘（重来会怎么做）。两个版本：30 秒电梯版、3 分钟深挖版。

1. 训练法：掏出手机录音讲 3 分钟，回听——这是成本最低的模拟面试；

2. 主语纪律：说“我”不说“我们”——面试官在确认的正是“你”干了什么。

⑦ **公开痕迹（public trail）**——让工作对面试官**可见**：

1. git 提交：一个提交一件事、message 说清“为什么”，永久告别 `update` 这类消息——`git log` 就是你的工作日历；
2. README 工程化：一句话定位 / 架构图 / 快速开始 / 测试与基准命令 / 设计文档（ADR）链接；
3. 加分现成资产：你的 `note/` 笔记与编译好的 PDF——“边做边写推导”是最稀缺的公开信号，挂进仓库或链接出去。

```
$ git log --oneline
…（希望长这样）

f3a9c21 fix: 补全 Del 并定 Get 未命中语义，kv_test.go:22/31 转绿
8be41d0 test: 为 Set 的 updated 语义先写考卷（预期红）
a17c5f2 docs: ADR-002 sorted_file 为何用有序段而不是哈希索引

// 避免：update / fix bug / 111
```

Listing 3: 理想的 `git log`：每个提交一件事、说清为什么；面试官扫一眼提交历史，就等于读到你半个月的工作日历

顺带一个现场观察：你当前 `git log --oneline` 只有一条 `init project`，而这 10 多天的全部工作（测试迁移、`note/`、`scratch/`）都还没提交——从“公开痕迹”的角度，它们对面试官尚不可见。今天就可以练一把：把它们分成几个语义提交，顺手感受一次“提交粒度”。

2.2.4. 考研特供：与 408 双向绑定

项目模块	408 考点（数据结构 / 操作系统）	面试考点
`kv.go` 内存表	散列表：冲突处理、装填因子	Go `map` 原理（10-07 已讲）
`sorted_file.go` 有序段	归并排序；文件 IO 与缓冲区	顺序写 vs 随机写、LSM 分层
`merge.go` 合并	二路 / 多路归并	数据库 compaction 策略
后续：B+ 树索引	B / B+ 树：插入分裂、范围查询（最热考点）	数据库索引（后端面试第一高频）
后续：缓冲池 / 并发	页面置换（LRU）、锁与信号量、死锁	Buffer Pool、MVCC、事务隔离

这张表的用法是双向的：

1. **复习 → 项目**：复习到“B+ 树的插入与分裂”，别只刷题——把分裂过程画进笔记（你会 CeTZ），并记一笔“这是 `sorted_file` 之后要实现的模块”；
2. **项目 → 复习**：写 `merge.go` 时翻出 408 的归并排序章节做对照——代码是考点的实体化，考点是代码的“为什么”。

为什么对你特别合适：db-go 的模块路线（哈希表 → 有序段 → 归并 → B+ 树 → 缓冲池 / 并发）几乎就是“数据结构 + 操作系统”的实体化过程。别人考研和项目是两件事，你这里可以合成一件事。

2.2.5. 时间盒：怎么塞进考研日程

1. **初试前（现在 → 12 月底）**：项目 = 每周一个固定时段（例如周日下午 4 小时），**到点就停**；每次结束前留 15 分钟做“蒸馏”——记 1 条决策、1 条故障档案、更新 1 次数字。素材只进不包装（全留在 `note/` 与 `git log`）；
2. **初试后 → 复试 / 求职前**：把素材组装成三件成品——简历条目（动词 + 技术 + 数字）、“20 问自答”清单、30 秒 / 3 分钟讲稿；
3. **红线**：项目的任何动作不得侵占数学与专业课的刚性时段——“证据动作”都设计成 5–15 分钟碎片，正是为了不碰这条红线。

2.3. 陷阱与易错

> ⚠️ **陷阱预警 最大的陷阱不是做少了，而是“只做不记”**——下面五条按杀伤力排序 注意它们全都不增加你的工作量，只改记录方式。

1. **只做不记**：事后补记必失真——故障细节的保质期只有几天；正确姿势是“现场 5 分钟”（本文八个动作的成本都按此设计）；
2. **简历堆名词**：写上去的每个词都是“约答题”——写了“并发”“索引”就会被追问三层，写不出来的宁可先别写；
3. **“我们”代替“我”**：课程 / 组队项目尤其注意——面试官要的是你个人的贡献边界，模糊的主语会被默认判定为“你没干什么”；
4. **数字造假或含糊**：编的数字一追问就塌；真实但小的数字（“约百 ns/op”）反而可信；
5. **项目膨胀**：广度没有上限，考研有——时间盒到点就停，宁可少一个模块，不留一张空头简历条目。

2.4. 关键点

> 📌 **核心要点 一句话**：动手实践是必要条件，不是充分条件；差额由四个“顺手动作”补齐——**决策日志**（为什么）、**量化数字**（多好）、**故障档案**（怎么坏、怎么修）、**叙事卡**（怎么讲）。每次实践收尾各花 5–15 分钟，考研不需要为它停下项目。

1. 优先级：③ 故障档案、① 决策日志先攒（它们记录“已经发生但正在遗忘”的事）；② 量化数字、⑥ 叙事卡随后（它们面向“未来要用”）；
2. 八个动作里 5 个是**记录**（①③④⑤⑦），3 个是**提炼**（②⑥⑧）——记录在现场，提炼在里程碑；
3. 共同点一句话：**留下可验证的痕迹**。

2.5. 变式检验

变式 1：项目没做完（或考研挤占到停滞）怎么办？——未完成不等于不可讲。面试官考察的是“你如何决策”，不是“你是否完工”。讲法：已完成的 60% 里我的关键取舍是什么 + 未

完成的 40% 我的路线图是什么 + 被什么打断、我如何取舍（“我把初试优先级放高”本身就是一次资源决策）。前提是你要能画出未完部分的路径图——这正是决策日志的用途。

变式 2：面试官盯住项目连问 20 分钟怎么办？——这是“深度”信号的考法。预案：写一张“20 问清单”，把自己项目最刁的问题写下来（示例：Go `map` 并发安全吗、你的 KV 怎么处理并发写、`os.File` 写一半程序崩了数据会怎样、`sorted_file` 的查找复杂度是多少、为什么不用 mmap……），每个问题准备 3 句话的回答 + 1 个能现场画的图。素材来源：你 `note/` 里已有的大量“为什么”（哈希、`defer`、泛型、TDD）可以直接复用。

变式 3：考研复试面试 vs 求职面试，动作要换吗？——动作相同，侧重不同：

面试类型	考官最想确认	你要端上的证据
复试（导师组）	学习能力、基础扎实度、可培养性	“我怎么查证、怎么推导、怎么实测”——你的 `note/` 笔记与 PDF 就是现成物证
求职（技术面）	工程能力：测试、性能、协作、工具链	基准数字、故障档案、提交历史、README 工程化

2.6. 同类归类

1. 与 **TDD** 同类：都是“先把证据留在现场，再动手”——红-绿-重构是**代码级证据**，本文八个动作是**职业级证据**（10-15 刚写过前者，今天补上后者）；
2. 与 **费曼笔记法** 同类：都是把隐性的思考过程（查了什么、排除了什么、实测了什么）显性化——你 `note/` 目录干的一直是这件事，今天只是给它加了一个“面试资产”的身份；
3. 与 **开源贡献** 同类：面试官对开源履历的信任来自“可公开验证”——你没法凭空开源一个项目，但可以让自己的仓库变成同样“可点开验证”的形态（动作 ⑦）。

> 💡 **背景与类比** 收束成一句：**凡是能留下可验证痕迹的实践，都比纯实践值钱**。你现有的工作流（实测 + 出处 + 推导的笔记）已经完成了最难的一半——今天增加的只是“记录方式”的调整。

2.7. 本次操作与当前状态

1. 新建本文件 `note/2026-10-16.typ`（当日归档：完整头部 + 当日起点 + 本主题），编译产出 `note/2026-10-16.pdf`；
2. **没有改动任何项目代码**：`internal/db/kv.go` 的两条待办（补 `Del`、改 `Get` 未命中）照旧；
3. 今天问答的“可动手”部分（如果想马上做一件，按成本排序任选）：
   1. （10 分钟）把 10-15 的两步改动落下去，跑 `go test -count=1 ./internal/db/` 转绿；
   2. （5 分钟）给 10-15 的 `vet` 静默事件贴上“故障档案 1”标签，格式用本文 ③ 的 6 行；
   3. （15 分钟）把现在未提交的工作分成 3–5 个语义提交，练一次“提交粒度”，同时让 10 天的劳动对面试官可见；
   4. （修绿后）把本文 ② 的 `bench_test.go` 模板放进 `internal/db/`，跑出你的第一个 `ns/op`；
4. 验证记录：`go build ./...` 报 `internal/db/kv.go:29:6: missing function body`（失败，符合预期——这正是待办）；本笔记经 `typst compile` 零错误（12 页，含 3 个代码清单）；

5. 一个小决定留给你：决策日志先留在 `note/`（零成本、顺手），还是马上在仓库建 `docs/decisions.md`？建议前者——等模块齐了再抽取。

Index of Listings

Listing 1 基准测试模板：`b.ResetTimer` 之前只做数据准备；`keys` 预生成而不是在循环里 `fmt.Sprintf`，否则测到的是“Sprintf + 查找”而不是“查找” 6
Listing 2 运行形状示意（数字以你机器实测为准）：学会读三列——`ns/op`（每次操作耗时）、`B/op`（每次操作字节数）、`allocs/op`（每次操作分配次数）；最后一行是竞态检测通过的样子 6
Listing 3 理想的 `git log`：每个提交一件事、说清为什么；面试官扫一眼提交历史，就等于读到你半个月的工作日历 8